

ResqDriver - Vehicle Assistance and Emergency Management Platform

Software Requirements Specification

Group 11

Group Members:

39323 – D.G.S. Wassana

39282 - A.K.M. Uyangoda

39316 – H.M.H.S. Medawatta

39140 – H.W.K.S. Wijerathna

39314 – D.D.S. De Zoysa

38946 – A.I. Amandi

39036 – H.I.W. Gamage

37924 – M.M.P. Deshitha

38910 – U.G.R. Ayesha

39073 – D.M.R.I.B. Dissanayaka

GitHub Frontend Repository: <https://github.com/kasunmihiran603/Resqdriver.git>

GitHub Backend Repository: <https://github.com/SahanWijerathna/Backend.git>

Table of Contents

1. Introduction
2. Overall Description
3. System Overview
4. Functional Requirements
5. Non-Functional Requirements
6. User Requirements
7. System Requirements
8. User Roles and Permissions
9. Software Architecture
10. Database Design
11. UML Diagrams
12. Data Flow Diagrams
13. Flowcharts
14. Screen Navigation
15. Wireframes
16. UI Design
17. API Documentation
18. Database Schema
19. Validation Rules
20. Business Rules
21. Security Requirements
22. Error Handling
23. Logging
24. Backup and Recovery
25. Testing
26. Future Enhancements
27. Limitations
28. Appendix

1. Introduction

1.1 Purpose

This document describes the requirements of our group project called **ResqDriver — Vehicle Assistance and Emergency Management Platform**. The purpose of the SRS is to explain what the system should do, how the different parts fit together, and how we plan to test it. We wrote this document before finishing development so that everyone in the team, and our supervisor, has the same understanding of the project.

The document is meant for:

- Our team members, to guide implementation.
- Our project supervisor and examiners, to evaluate the project.
- Testers (peer reviewers), to write test cases from the requirements.
- Future students, if they extend the project in a later course.

1.2 Scope

ResqDriver is a web-based platform that helps drivers who have a vehicle problem on the road. For example, a flat tyre, a dead battery, an engine issue, or an accident that needs a tow truck. The platform connects four types of users:

- **Drivers**, who need help.
- **Garages**, who fix vehicles.
- **Towing operators**, who tow vehicles when they cannot be fixed on the spot.
- **Administrators**, who manage the platform.

What the system will do

- Let users register and log in based on their role.
- Let drivers register their vehicles and create assistance or towing requests with GPS location.
- Show pending requests to the correct type of provider (garages see repair jobs, towing operators see towing jobs).
- Let providers accept a request and progress it through a defined status flow.
- Automatically calculate the cost based on distance and the provider's per-km rate.
- Send in-app notifications when a request status changes.
- Let drivers make a payment after service and download a receipt (PDF).
- Give admins a dashboard, user management, and CSV export.

What the system will *not* do (for now)

- No real payment gateway — payment is simulated for this version.
- No real SMS or push notifications.
- No native mobile app — we build a responsive web app.
- No live GPS hardware tracking of vehicles.
- No real-time WebSocket updates in this version (the frontend refreshes on demand).

1.3 Definitions and Abbreviations

Term	Meaning
SRS	Software Requirements Specification
API	Application Programming Interface
REST	Representational State Transfer
JWT	JSON Web Token (used to keep a user logged in)
CRUD	Create, Read, Update, Delete
DTO	Data Transfer Object
ORM	Object Relational Mapping (JPA / Hibernate)
GPS	Global Positioning System (latitude / longitude)
ERD	Entity Relationship Diagram
DFD	Data Flow Diagram
UML	Unified Modeling Language
RBAC	Role-Based Access Control
SPA	Single Page Application
ETA	Estimated Time of Arrival
Dispatch Cost	The service fee, based on distance and the provider's rate per km
Platform Commission	10% of the dispatch cost, kept by the platform
Driver / User	The customer who needs assistance
Garage	A registered repair shop
Towing Operator	A registered tow-truck company
Admin	Platform administrator

1.4 References

We used the following documents and websites while preparing this SRS:

- Spring Boot documentation — <https://spring.io/projects/spring-boot>
- React documentation — <https://react.dev>
- MySQL 8 Reference Manual — <https://dev.mysql.com/doc>
- Tailwind CSS — <https://tailwindcss.com>
- Notes from the Software Engineering and Database Management courses.

2. Overall Description

2.1 Product Perspective

ResqDriver is a brand-new system built from scratch for our group project. It is a three-tier web application:

- A **React** single-page frontend (built with Vite).
- A **Spring Boot** backend that provides a REST API.
- A **MySQL** database.

The system also uses an **SMTP mail server** in the background to send password-reset emails. If the mail server is not available in development, the reset link is printed to the server console instead.

2.2 Main Product Functions

A short summary of what the system does:

#	Feature	Short Description
1	Registration and Login	Users sign up and log in using email, password, and role. JWT is used to keep them logged in.
2	Vehicle Management	Drivers can add, edit, and remove their vehicles.
3	Assistance Request	Drivers can create a roadside repair request with issue category and GPS location.
4	Towing Request	Drivers can create a towing request with pickup and destination.
5	Status Tracking	Drivers can track their request through a status timeline.
6	Provider Acceptance	Garages and towing operators can accept and update jobs.
7	Cost Calculation	Dispatch cost, 10% commission, and provider share are calculated automatically.
8	Payment	Drivers confirm payment after service. Transactions are recorded.
9	Notifications	In-app notifications for key events.
10	Admin Panel	User management, analytics, and CSV export.

2.3 User Classes

User Class	Description	Technical Skill Needed
Driver (User)	Vehicle owner who needs help	Basic smartphone / web skills
Garage	Repair shop that provides services	Basic to intermediate
Towing Operator	Tow-truck company	Basic to intermediate

User Class	Description	Technical Skill Needed
Administrator	Platform staff	Intermediate

2.4 Operating Environment

- **Client side:** Any modern browser (Chrome, Edge, Firefox, Safari) with JavaScript enabled and internet.
- **Server side:** Java 17 with Spring Boot (embedded Tomcat) on Windows or Linux.
- **Database:** MySQL 8.
- **Development tools:** VS Code / IntelliJ IDEA, Node.js + npm, Maven.

2.5 Constraints

- The backend must use **Spring Boot** with **Java 17** and **Maven** because they were taught in our course.
- The frontend must use **React 19** with **Vite** and **Tailwind CSS**.
- The database must be **MySQL**.
- Authentication must use **JWT tokens** (not server-side sessions).
- The system supports four fixed roles: *user*, *garage*, *towing*, *admin*.
- The project must be finished within one semester by a small team.

2.6 Assumptions

- The user has an internet connection and a GPS-enabled browser/device.
- A MySQL server is running before the backend starts (the backend creates tables on first run).
- An SMTP mail server is available for password reset. If not, the reset link is logged to the console.
- Distances are calculated using the Haversine formula (straight-line distance), not actual road routes.

3. System Overview

ResqDriver digitises the roadside-assistance process. When a driver has a problem, they open the app and pick an issue category — battery, engine, tyre, overheating, accident, or unknown — or they request a tow truck. They select their vehicle, share their GPS location, and submit the request.

The request appears as **pending** to nearby garages (for repairs) or towing operators (for tows). A provider accepts the request. The system then automatically calculates the **dispatch cost** using the distance and the provider's rate per km, works out the **10% platform commission**, and gives the rest to the provider.

For a garage, the next step is to assign a technician. The status then progresses through *on_the_way* → *repair_in_progress* → *completed*. The driver can watch this timeline live on their Track page.

When the job is done, the driver confirms payment. The status becomes *paid* and both the driver and the provider can see a transaction record.

Admins have a top-down view of the whole platform. They can see all users, all requests, all transactions, suspend accounts, and export data as CSV.

4. Functional Requirements

Functional requirements describe what the system should do. We grouped them into nine modules.

4.1 Authentication and Account Management

What it does: Handles register, login, profile, logout, and password reset.

ID	Requirement
FR-1.1	Users can register with name, email, password, phone, and role. Passwords are hashed with BCrypt before saving.
FR-1.2	If the role is <i>garage</i> , the user can also add services and technicians during registration.
FR-1.3	Users can log in with email, password, and role. The backend gives back a signed JWT that lasts 7 days.
FR-1.4	Users can view their own profile at <i>GET /api/auth/me</i> .
FR-1.5	Users can update their profile at <i>PUT /api/auth/profile</i> .
FR-1.6	Users can request a password reset. A token is created that expires in 15 minutes and is emailed to the user.
FR-1.7	Users can reset their password using a valid token.
FR-1.8	Users can log out. The JWT cookie is cleared.

Validation: Email must be valid and unique. Password must be at least 6 characters. Login role must match the registered role.

Errors: Duplicate email → 400. Wrong password → 401. Expired token → 400.

4.2 Vehicle Management

What it does: Lets a driver keep a list of their vehicles.

ID	Requirement
FR-2.1	A driver can add a new vehicle (make, model, year, plate, insurance).
FR-2.2	A driver can edit an existing vehicle.
FR-2.3	A driver can delete a vehicle.
FR-2.4	The list of a driver's vehicles is shown in the request wizard for selection.

Validation: Make, model, and plate are mandatory. Year must be a 4-digit number like *2020*.

Business rule: A driver must have at least one vehicle before creating a request.

4.3 Assistance Request (Garage)

What it does: Lets a driver ask for roadside repair, and lets garages accept and finish the job.

ID	Requirement
FR-3.1	A driver can submit a request. Status defaults to <i>pending</i> , payment status to <i>unpaid</i> .
FR-3.2	The system calculates the distance between the driver and the garage using the Haversine formula.
FR-3.3	Garages can see pending requests near them.
FR-3.4	A garage can accept a request. The system sets <i>garageId</i> , changes status to <i>accepted</i> , and computes <i>dispatchCost</i> , <i>platformCommission</i> (10%), and <i>providerShare</i> (90%).
FR-3.5	The garage can assign a technician. Status becomes <i>technician_assigned</i> .
FR-3.6	Status can progress to <i>on_the_way</i> → <i>repair_in_progress</i> → <i>completed</i> .
FR-3.7	A notification is automatically created for the driver when a request is accepted.
FR-3.8	The driver can see the live status of their own request.
FR-3.9	Requests can be filtered by category.

Business rules:

- Status must move in the defined order. Invalid transitions are rejected.
- $\text{Dispatch Cost} = (\text{distance in km} \times \text{provider's ratePerKM}) \div 300$
- $\text{Platform Commission} = \text{Dispatch Cost} \times 10\%$
- $\text{Provider Share} = \text{Dispatch Cost} - \text{Platform Commission}$

4.4 Towing Request

What it does: Same idea as assistance requests but for towing, with pickup and drop-off locations.

ID	Requirement
FR-4.1	A driver can submit a towing request with pickup and destination GPS.
FR-4.2	Towing operators see pending towing requests.
FR-4.3	A towing operator can accept a job. Cost is calculated the same way.
FR-4.4	Status uses the same lifecycle.
FR-4.5	The pickup → destination route info is shown to both the driver and the operator.

4.5 Technician Management

What it does: Lets a garage keep a list of technicians.

ID	Requirement
FR-5.1	A garage can add a technician (default status <i>available</i>).
FR-5.2	A garage can edit or remove a technician.
FR-5.3	A garage can assign an available technician to an accepted request.

4.6 Service Catalogue

What it does: Lets a garage list the repair services it offers.

ID	Requirement
FR-6.1	A garage can add, edit, and delete services.
FR-6.2	Services appear on the garage profile.

4.7 Transactions and Payment

What it does: Records payments and shows transaction history.

ID	Requirement
FR-7.1	When a request is <i>completed</i> , the driver can confirm payment. The request's <i>paymentStatus</i> becomes <i>paid</i> .
FR-7.2	Users can see their own transactions. Admins can see all.
FR-7.3	Drivers can download a PDF summary of a transaction.

Business rule: Payment can only be made after the status is *completed*. Payment cannot be made twice.

4.8 Notifications

ID	Requirement
FR-8.1	A notification is automatically created when a request is accepted.
FR-8.2	Users see their notifications, newest first.
FR-8.3	Users can mark all notifications as read.
FR-8.4	Users can delete individual notifications.

4.9 Administration

ID	Requirement
FR-9.1	An admin can list all users, filtered by role.
FR-9.2	An admin can suspend or reinstate a user.
FR-9.3	An admin (or the user themselves) can soft-delete an account.
FR-9.4	An admin can see all requests and transactions.
FR-9.5	An admin can export all requests as a CSV file.
FR-9.6	The admin dashboard shows counts, active/completed incidents, and total platform commission.

Business rule: Suspended users cannot log in, but their history is kept.

5. Non-Functional Requirements

Category	Requirement
Performance	API responses should return in under 2 seconds under normal load (up to about 50 users at once).
Security	Passwords are stored using BCrypt hashing. Authentication uses signed JWTs. Sensitive endpoints require the correct role.
Reliability	The system correctly saves all committed data. MySQL InnoDB gives ACID guarantees.
Availability	We target 99% availability during demo and evaluation, not 24/7 production.
Scalability	The stateless REST API allows us to scale the backend behind a load balancer in future.
Maintainability	The backend code is organised in clear layers: controller, service, repository, model, dto.
Usability	The UI is consistent (uses Tailwind CSS), supports light and dark themes, and shows form validation feedback clearly.
Accessibility	Labels and colour contrast are appropriate. Colour alone is not used to show status — icons and text are also used.
Compatibility	The frontend works on the latest Chrome, Edge, Firefox, and Safari, on both desktop and mobile.
Responsiveness	Layout adjusts for screens smaller than 768px (sidebar becomes bottom nav, tables become cards).

6. User Requirements

These are user stories, from the point of view of each type of user.

ID	User Story
UR-1	As a driver, I want to register my vehicles so I can quickly select them when I need help.
UR-2	As a driver, I want to send an assistance request with my location so a nearby garage can come to me.
UR-3	As a driver, I want to request a tow truck if my vehicle cannot be fixed on the spot.
UR-4	As a driver, I want to track the live status of my request so I know when help will arrive.
UR-5	As a driver, I want to see the cost first, then pay after the job is done.
UR-6	As a garage owner, I want to see nearby requests and accept the ones I can service.
UR-7	As a garage owner, I want to assign a technician and update the job status.
UR-8	As a garage owner, I want to see my earnings and job history.
UR-9	As a towing operator, I want to see and accept tow jobs with pickup and drop-off.
UR-10	As an admin, I want to see all users and requests, and suspend problem accounts.
UR-11	As any user, I want to reset my password if I forget it.
UR-12	As any user, I want to get a notification when my request status changes.

7. System Requirements

7.1 Hardware (Development / Demo)

Component	Minimum
Processor	Dual-core 2 GHz
RAM	8 GB dev machine / 2 GB server
Storage	5 GB free
Network	Broadband internet

7.2 Software

Layer	Technology
Frontend Runtime	Node.js $\geq$ 18, npm
Frontend Framework	React 19, Vite

Layer	Technology
Frontend Libraries	React Router, React Hook Form, Tailwind CSS, Framer Motion, Leaflet (maps), Recharts (charts), Axios, jsPDF, lucide-react
Backend Runtime	Java 17 (JDK)
Backend Framework	Spring Boot 3.4 (Web, Data JPA, Security, Mail, Validation)
Build Tool	Maven
Auth	JWT 0.11.5
Database	MySQL 8
IDE	VS Code / IntelliJ IDEA
Version Control	Git

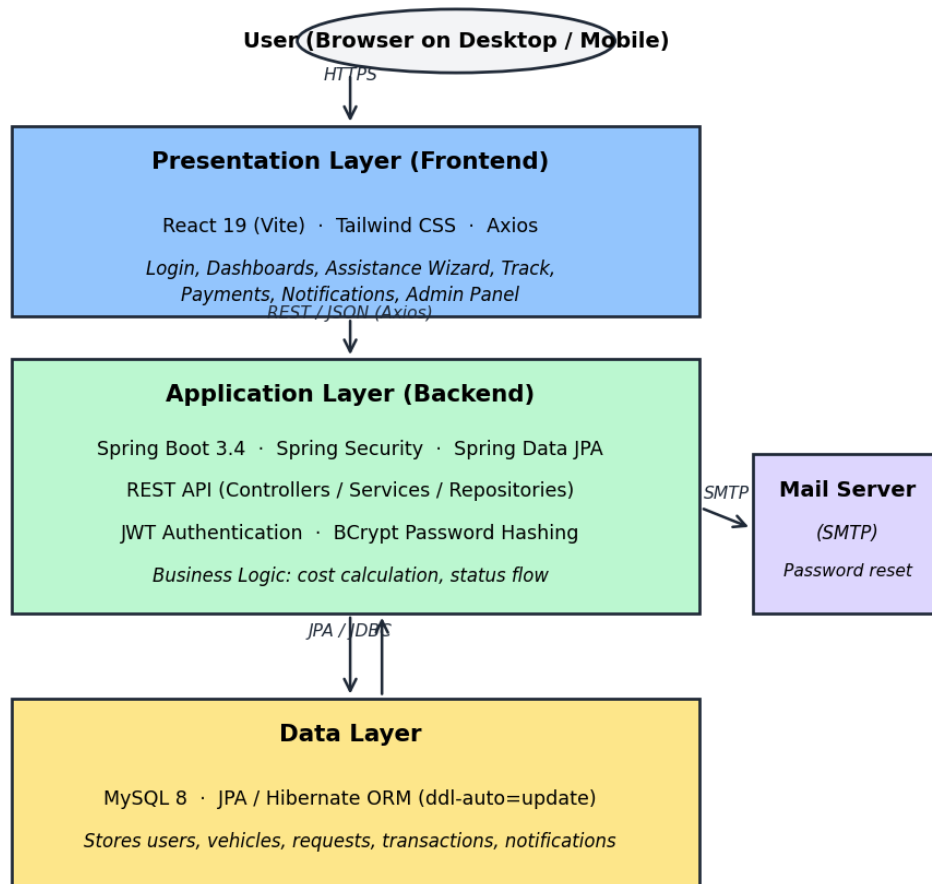
8. User Roles and Permissions

The table shows what each role can and cannot do. ✓ means allowed, ✗ means not allowed.

Action	Driver	Garage	Towing	Admin
Register / Login	✓	✓	✓	✓
Manage own profile	✓	✓	✓	✓
Manage own vehicles	✓	✗	✗	✗
Create assistance / towing request	✓	✗	✗	✗
View own requests	✓	✗	✗	✓ (all)
View pending requests	✗	✓ (repair)	✓ (tow)	✓ (all)
Accept / update request	✗	✓	✓	✓
Manage technicians	✗	✓	✗	✗
Manage services	✗	✓	✗	✗
Make payment	✓	✗	✗	✗
View transactions	✓ (own)	✓ (own)	✓ (own)	✓ (all)
Manage notifications	✓	✓	✓	✓
Suspend / delete users	✗	✗	✗	✓
Export requests (CSV)	✗	✗	✗	✓
View platform analytics	✗	✗ (own)	✗ (own)	✓

9. Software Architecture

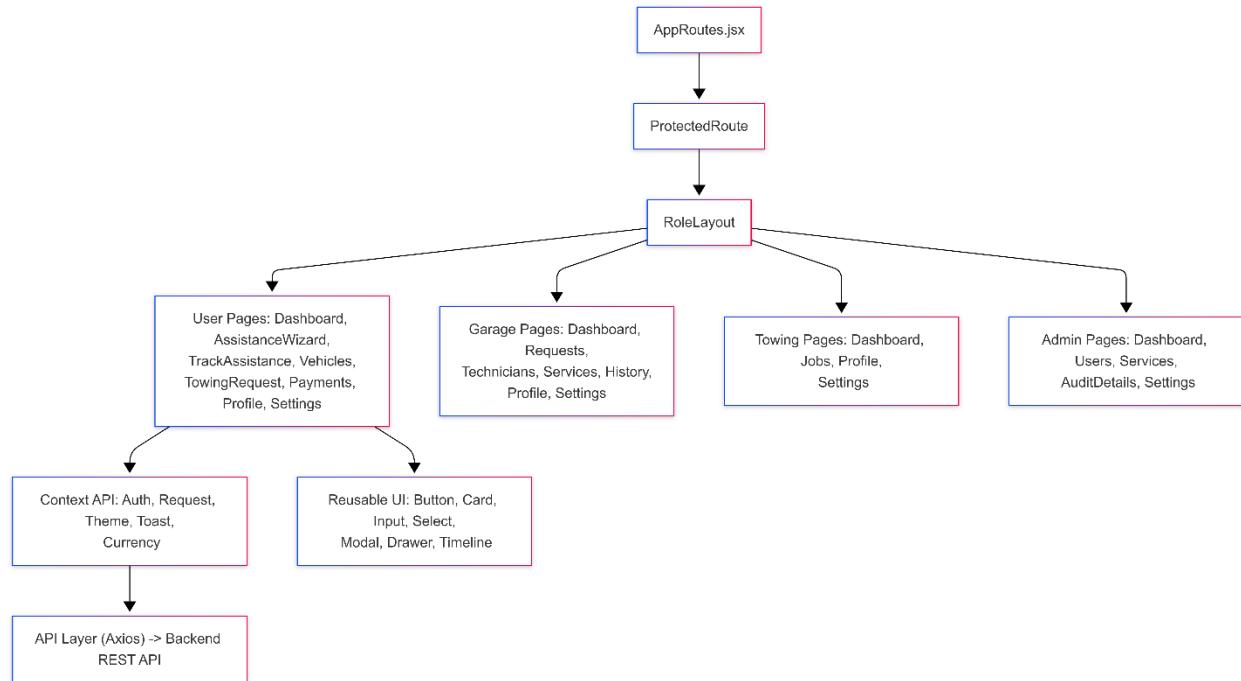
The system uses a standard three-tier architecture. The diagram below shows the layers and how they talk to each other.



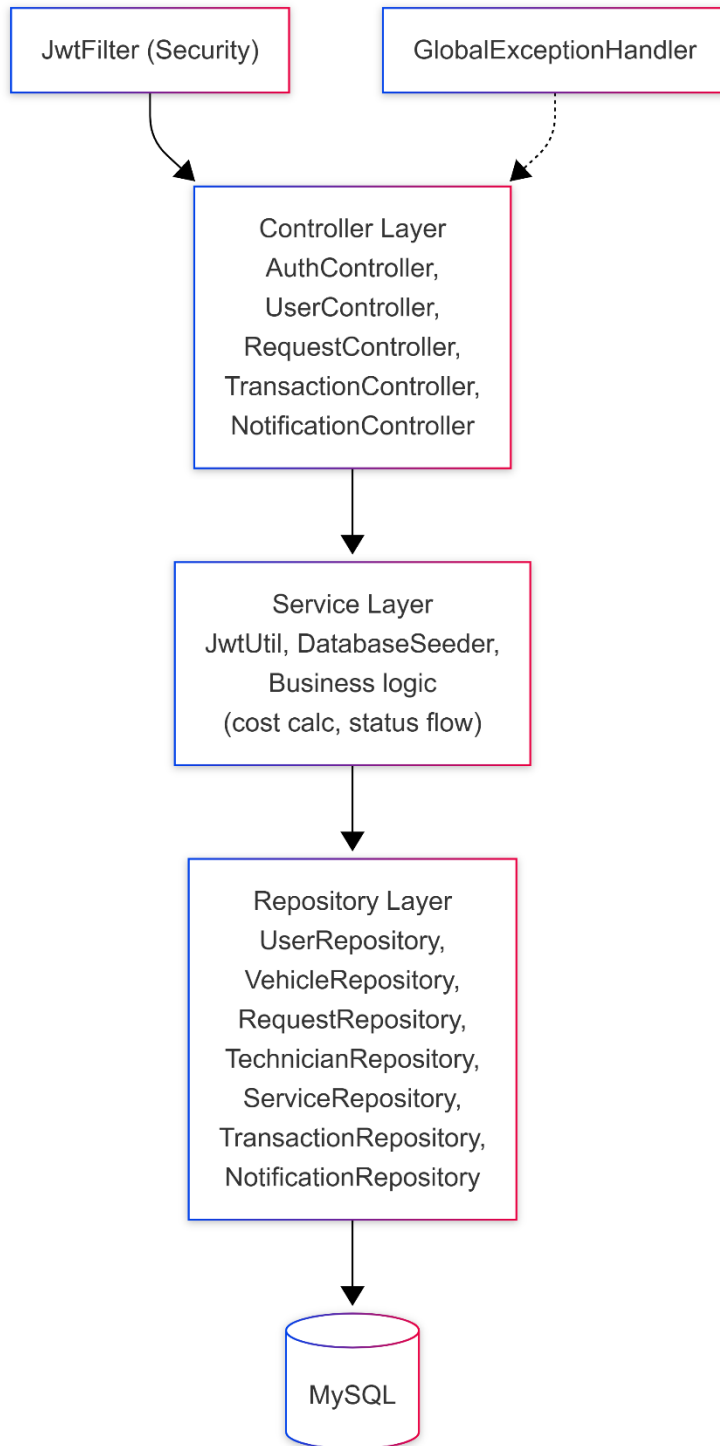
How it works:

- The **Presentation Layer** is the React frontend the user sees.
- The **Application Layer** is the Spring Boot backend. It handles authentication, business logic, and data access.
- The **Data Layer** is MySQL. Spring Data JPA (Hibernate) talks to it.
- An external **SMTP** mail server is used only for password-reset emails.

9.1 Frontend Architecture



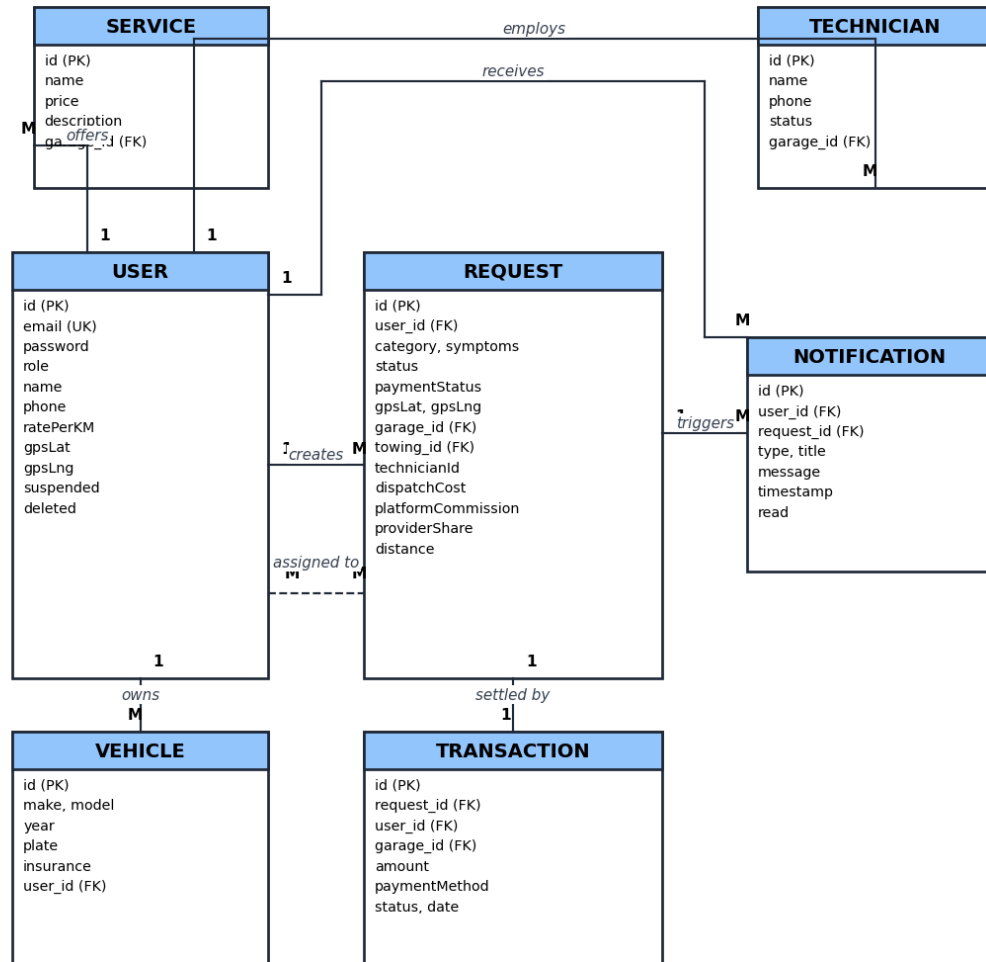
9.2 Backend Architecture



10. Database Design

10.1 ER Diagram

The diagram below shows the main tables (entities) and how they are connected.



10.2 Entity Descriptions

Entity	Purpose
User	Stores every account (driver, garage, towing, admin) in one table, using the <i>role</i> column to tell them apart.
Vehicle	A vehicle owned by a driver.
Service	A repair service offered by a garage, with name/price.
Technician	A staff member of a garage.
Request	The central record for one assistance or towing job, including its full lifecycle.
Transaction	A payment record linked to a completed request.
Notification	An in-app message sent to a user.

10.3 Relationships

Relationship	Type	Notes
User → Vehicle	1 to Many	A driver owns many vehicles.
User → Service	1 to Many	A garage offers many services.
User → Technician	1 to Many	A garage employs many technicians.
User → Request	1 to Many	A driver creates many requests.
Request → Transaction	1 to 1	A completed request has one transaction.
Request → Notification	1 to Many	A request can generate several notifications.
User → Notification	1 to Many	A user receives many notifications.

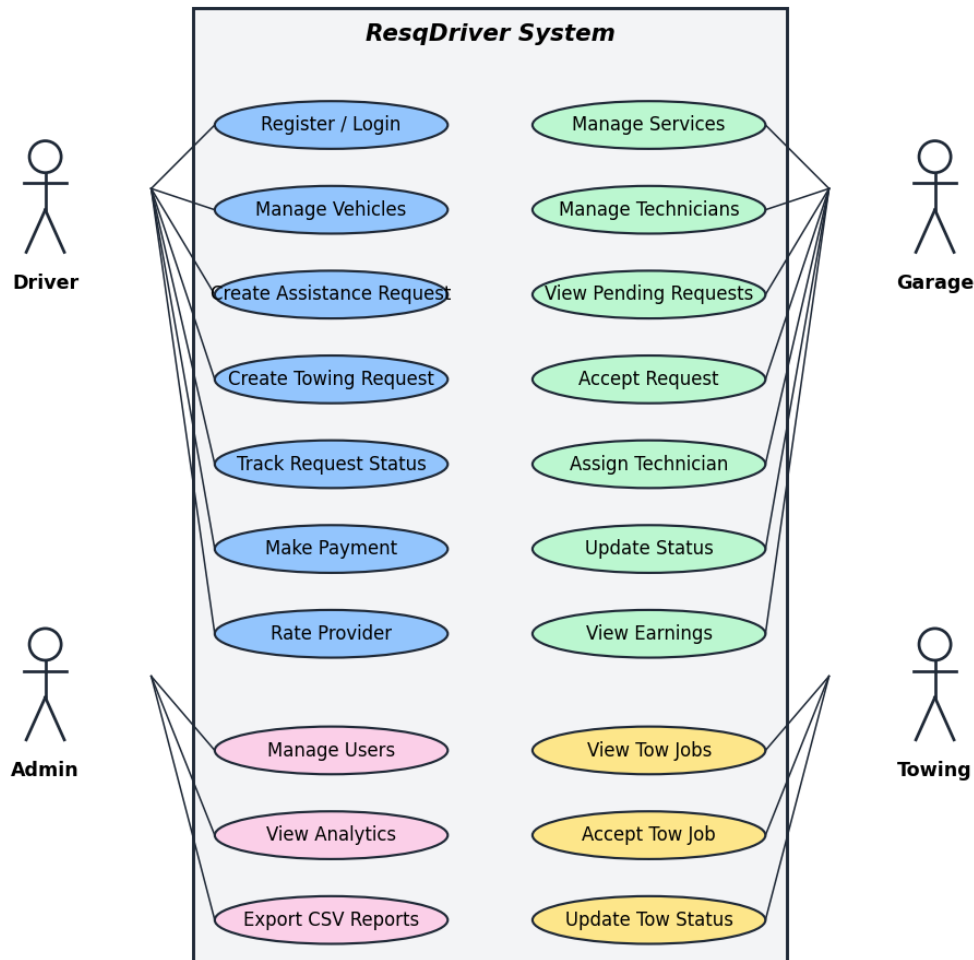
10.4 Primary Keys

All tables use a String primary key (*id*) generated as a UUID.

11. UML Diagrams

11.1 Use Case Diagram

The use case diagram shows the main actions each type of user can perform.



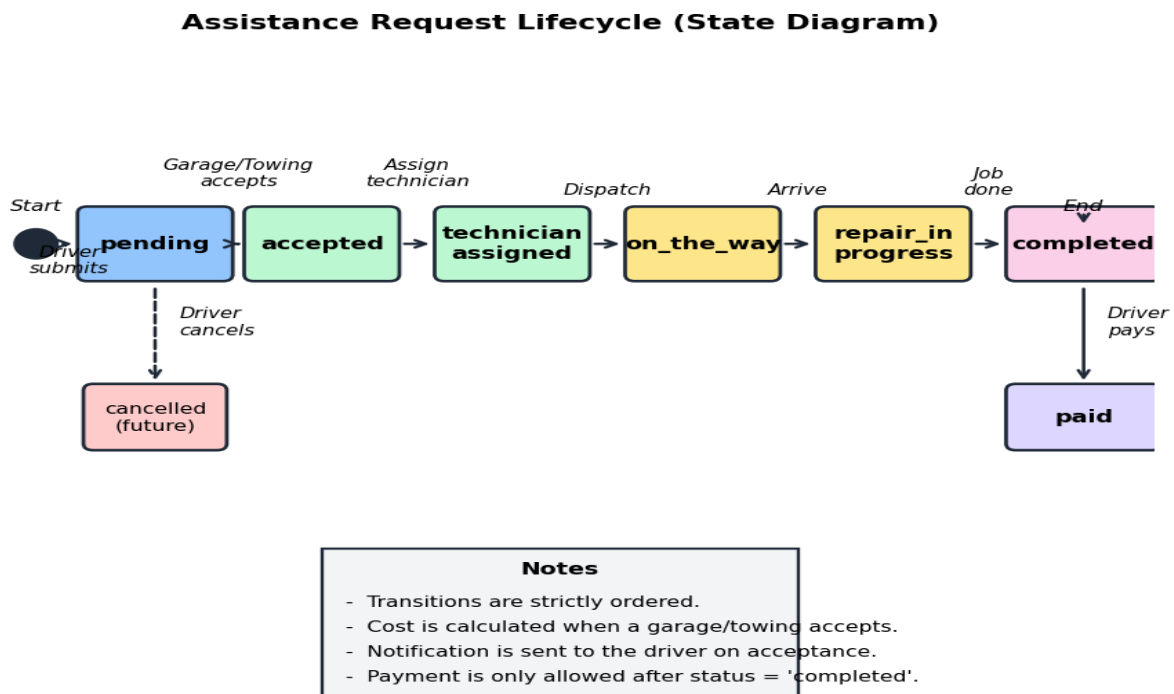
11.2 Use Case Descriptions

Use Case	Actor	Precondition	Main Flow	Postcondition
Create Assistance Request	Driver	Logged in, has a vehicle	Pick category → pick symptoms → pick vehicle → set location → submit	Request created with status <i>pending</i>
Accept / Update Request	Garage / Towing	Logged in, request is pending or assigned to them	View request → accept → assign technician → update status	Status moves through the lifecycle

Use Case	Actor	Precondition	Main Flow	Postcondition
Make Payment	Driver	Request status = <i>completed</i>	View cost → pick payment method → confirm	Transaction saved, <i>paymentStatus</i> = <i>paid</i>
Manage Users	Admin	Logged in as admin	View list → filter → suspend / delete	User record updated
Export CSV	Admin	Logged in as admin	Click export	CSV file downloaded

11.3 State Diagram — Request Lifecycle

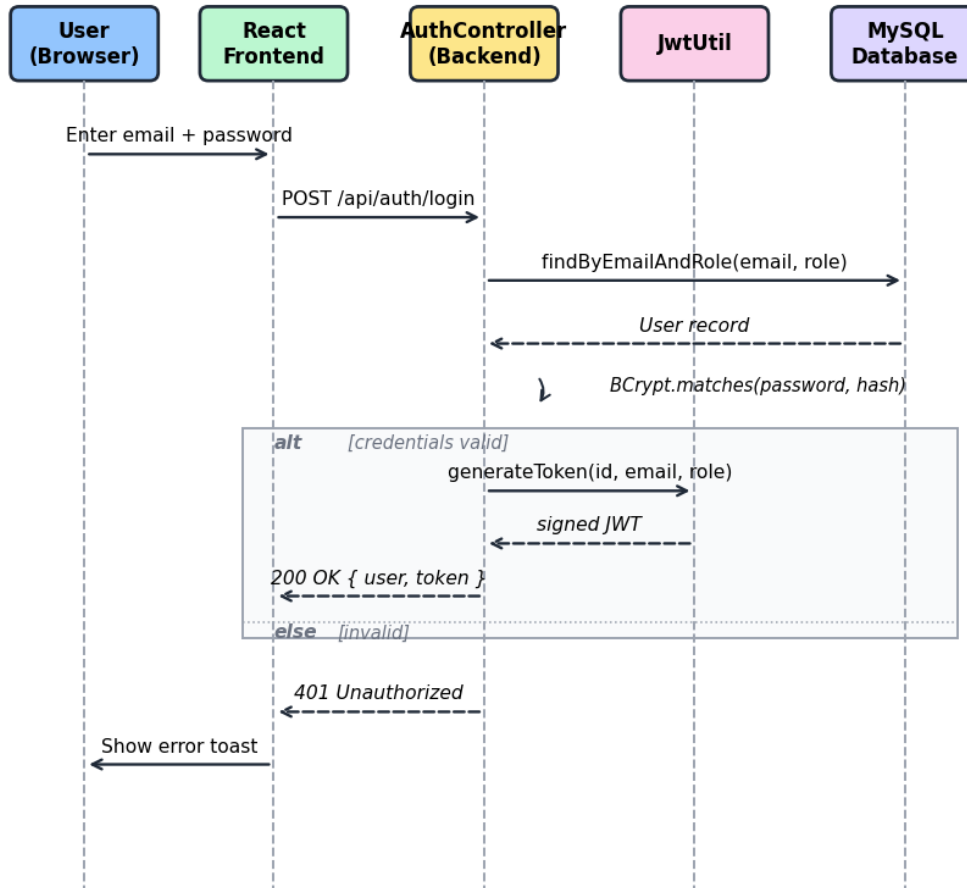
The diagram below shows how an assistance request moves through its states.



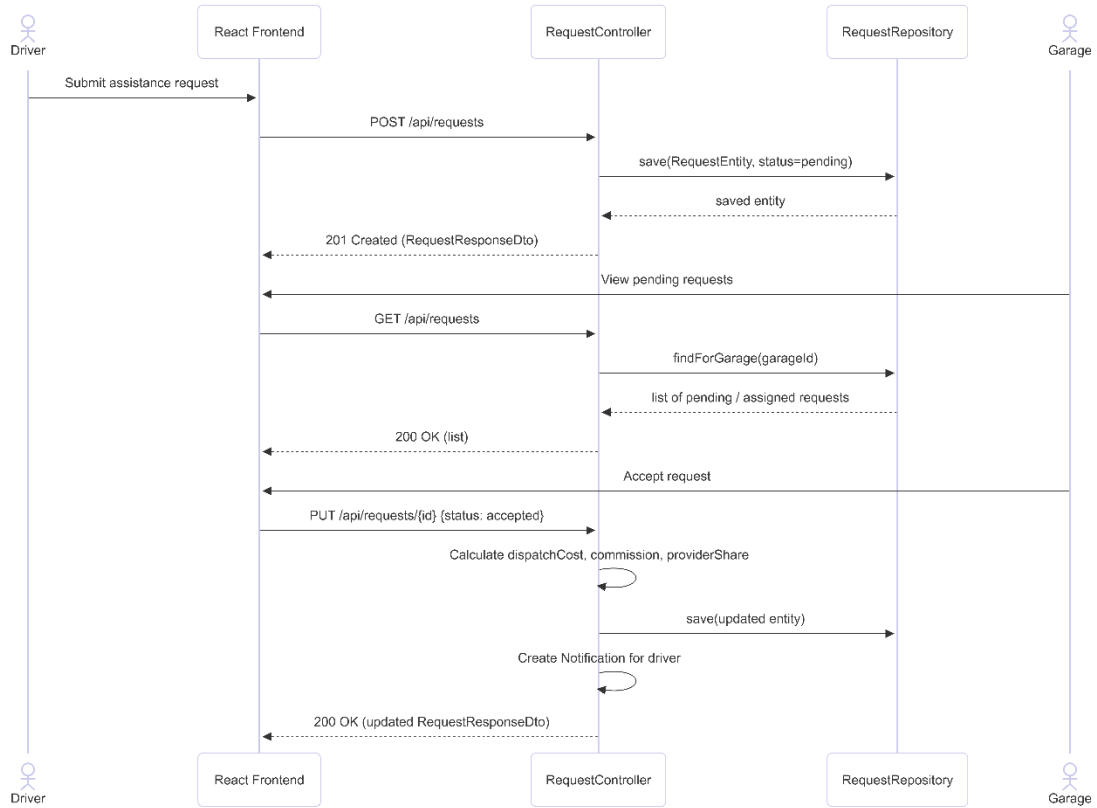
1.4 Sequence Diagram — Login

The next diagram shows what happens when a user logs in with JWT.

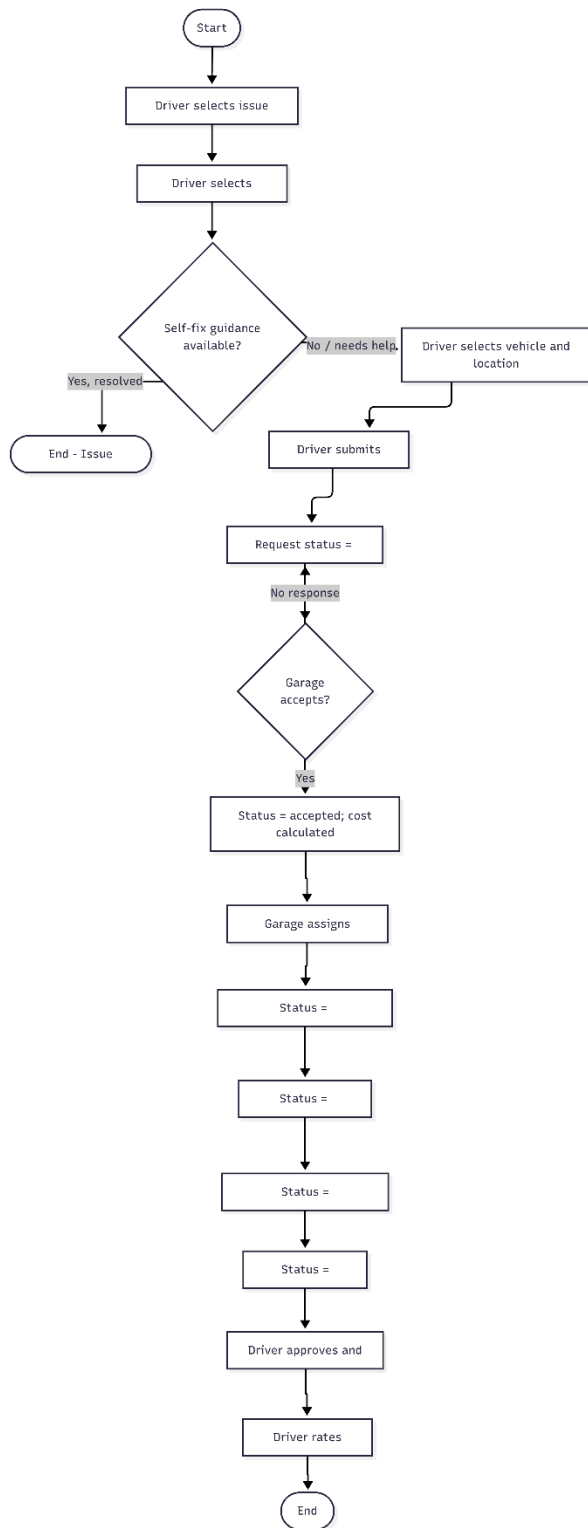
Sequence Diagram - Login with JWT



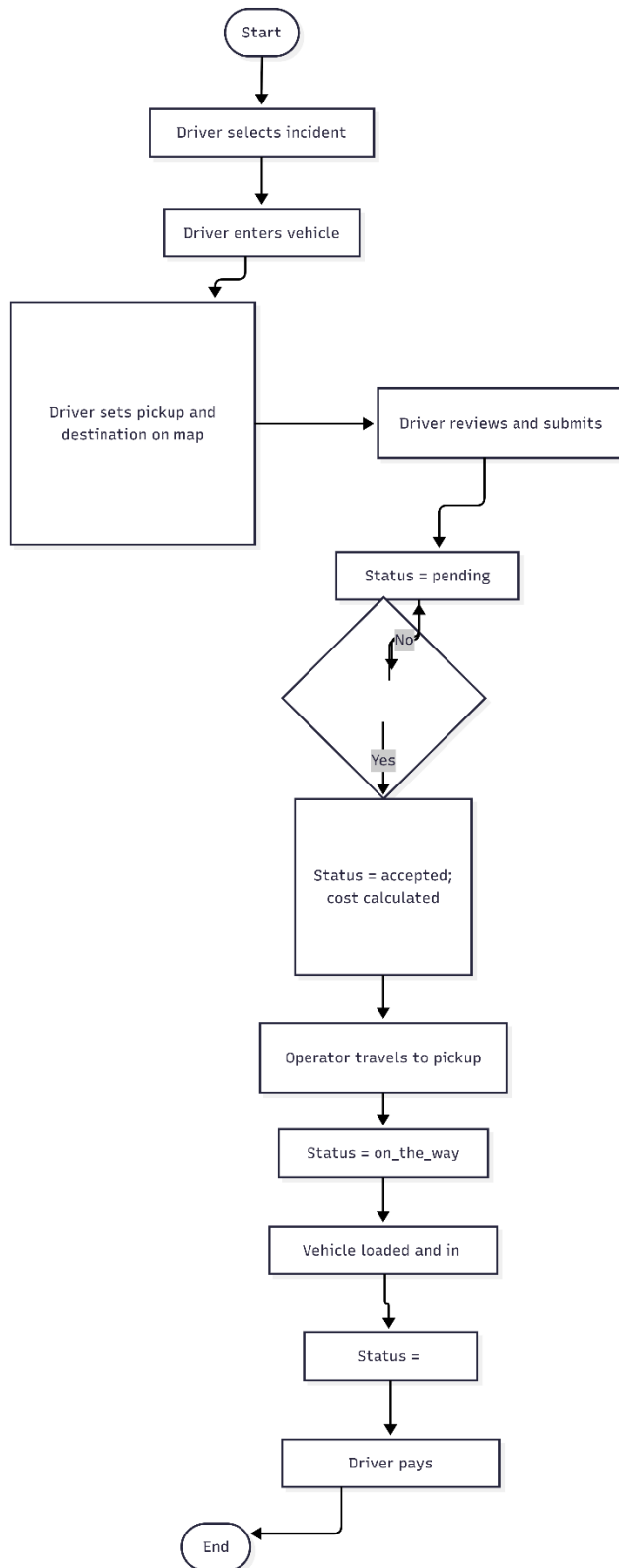
11.5 Sequence Diagram — Create and Accept Request



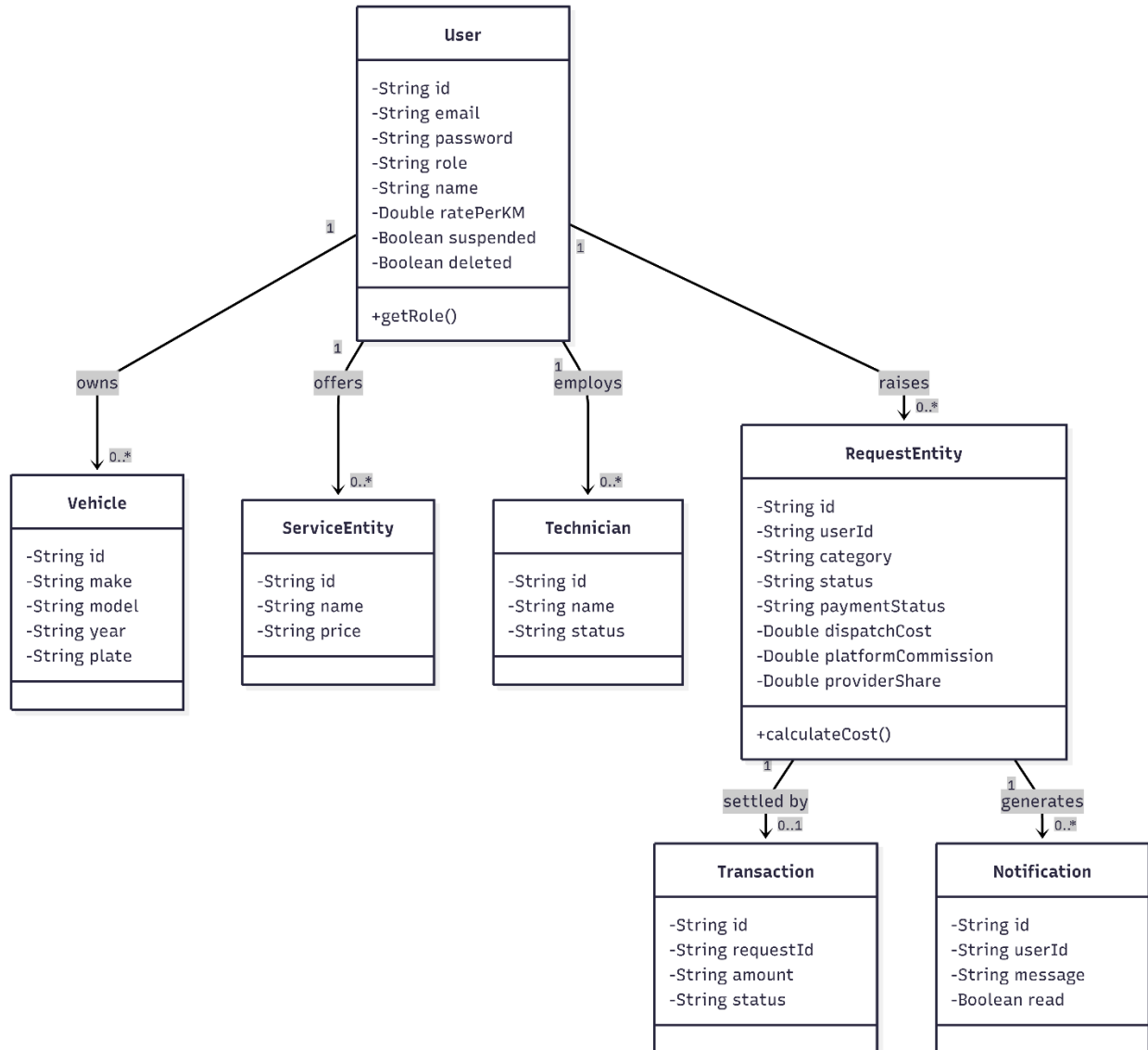
11.6 Activity Diagram — Assistance Request Flow



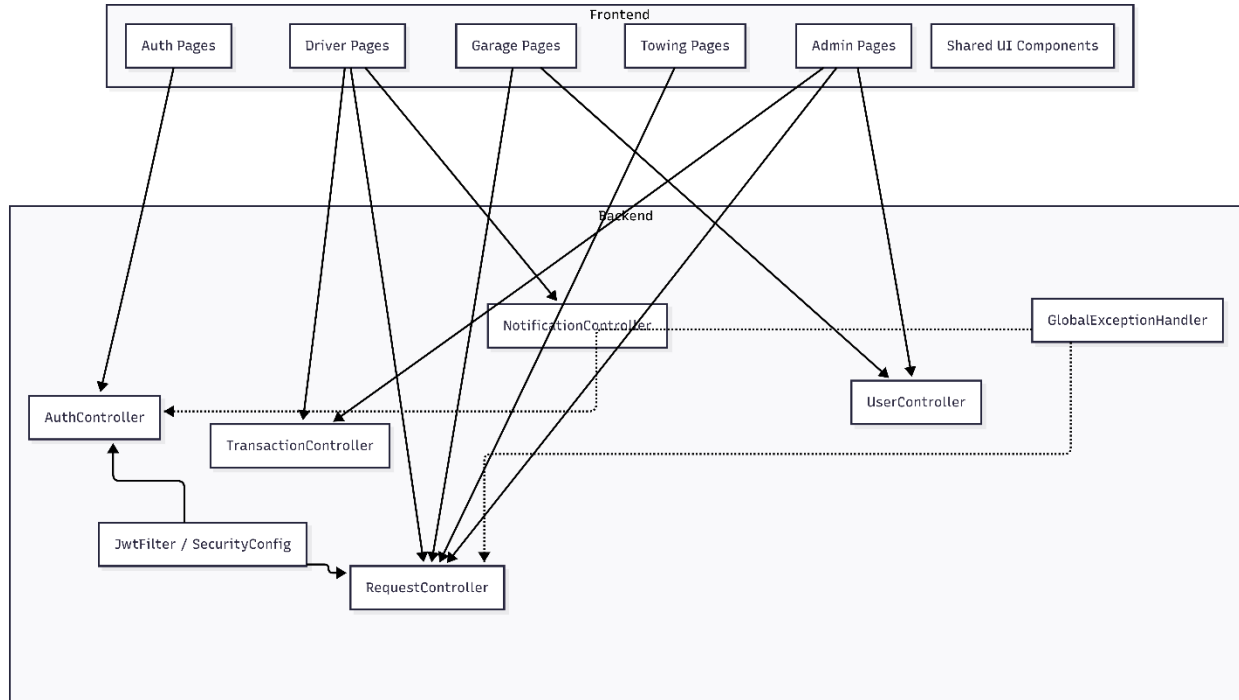
11.7 Activity Diagram — Towing Request Flow



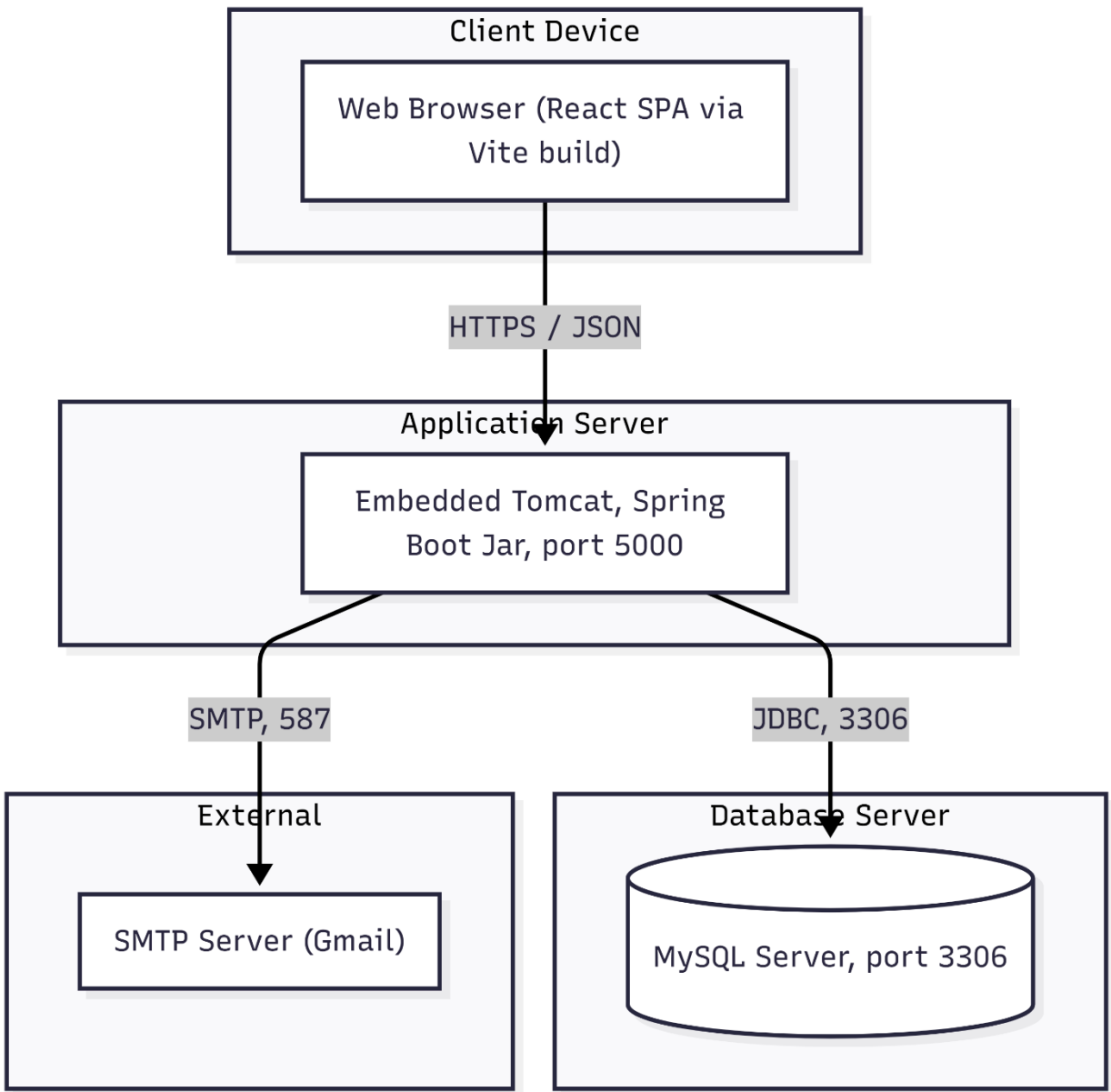
11.8 Class Diagram



11.9 Component Diagram

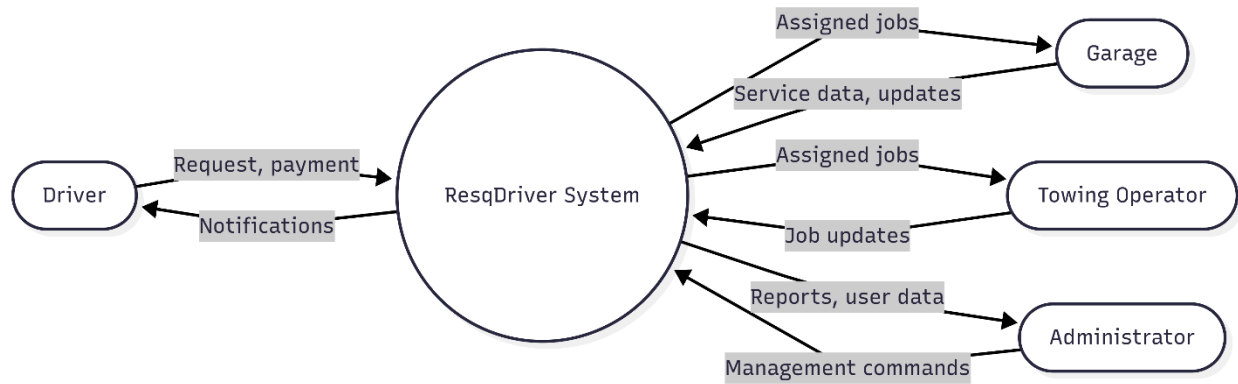


11.10 Deployment Diagram



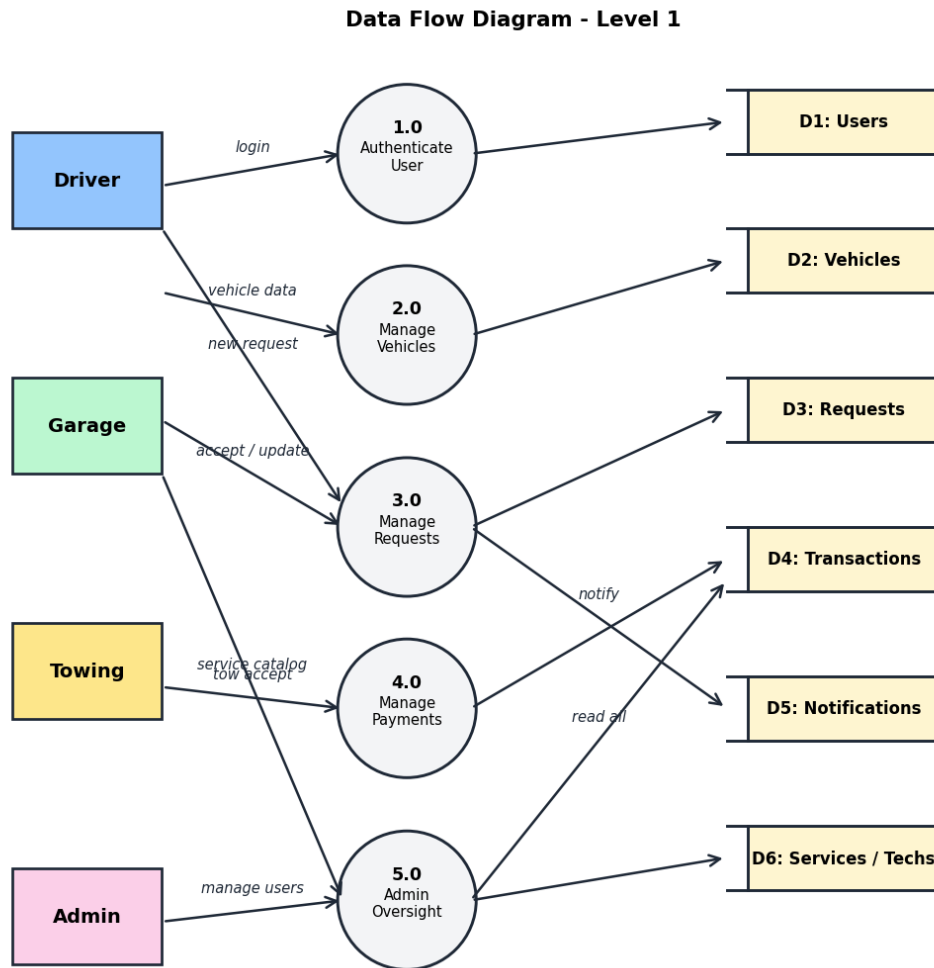
12. Data Flow Diagrams

12.1 DFD Level 0 (Context)



12.2 DFD Level 1

The diagram below shows how data flows between users, processes, and data stores.



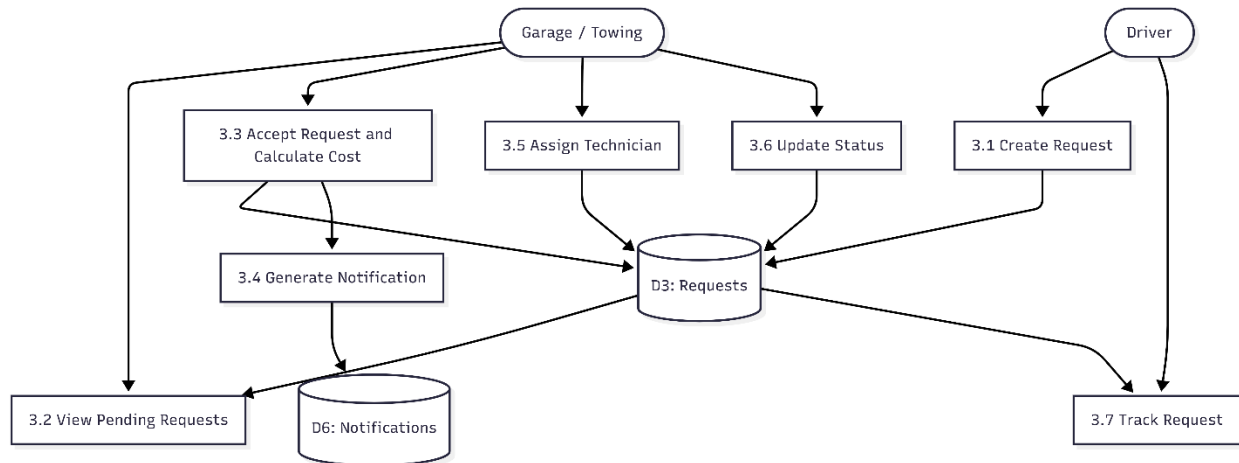
Processes:

1. **Authenticate User** — verifies email and password, issues JWT.
2. **Manage Vehicles** — CRUD for vehicles.
3. **Manage Requests** — creating, accepting, updating requests.
4. **Manage Payments** — creating transactions.
5. **Admin Oversight** — user management, reports, CSV export.

Data Stores:

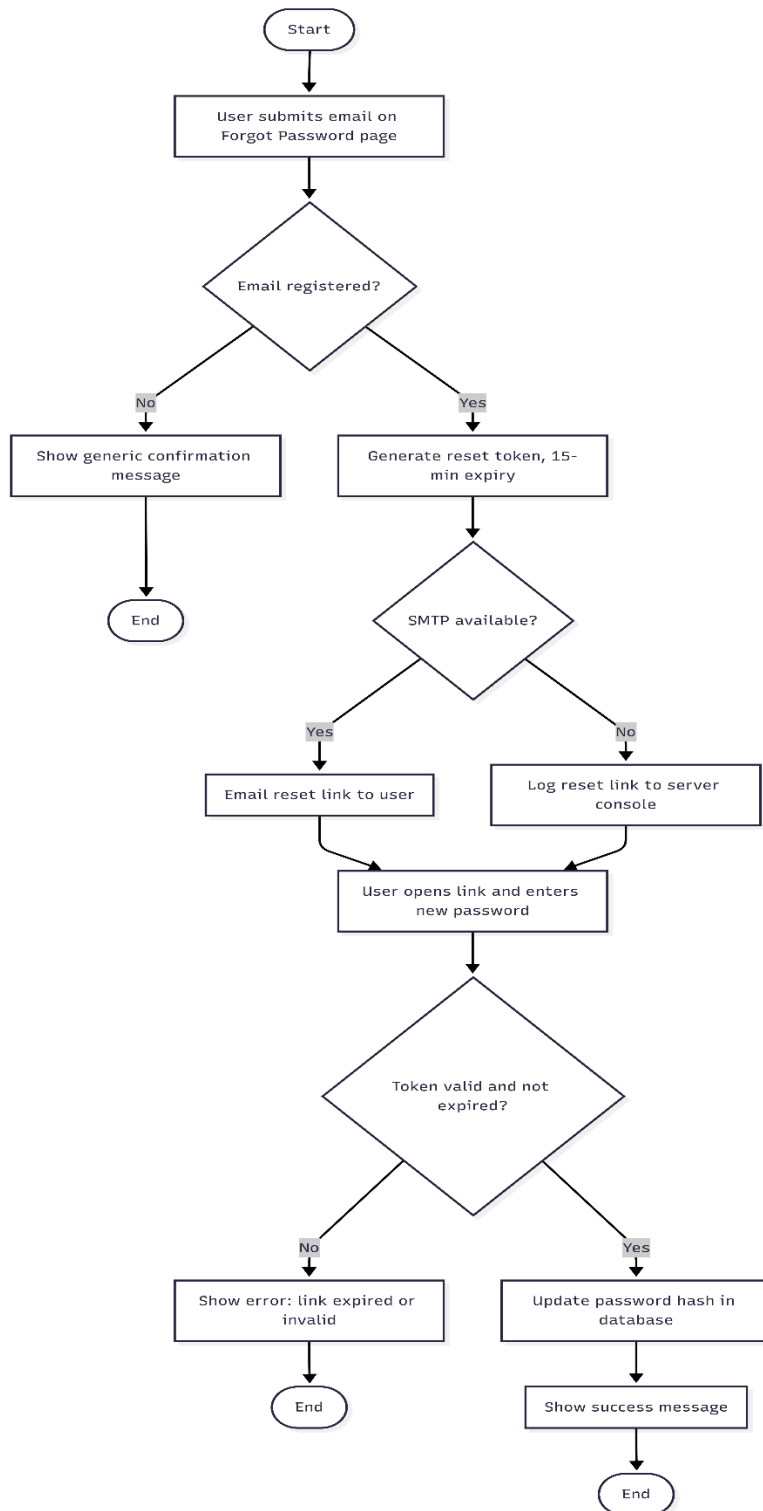
- D1 Users
- D2 Vehicles
- D3 Requests
- D4 Transactions
- D5 Notifications
- D6 Services / Technicians

12.3 DFD Level 1 — Manage Requests Sub-Process (Mermaid Source)



13. Flowcharts

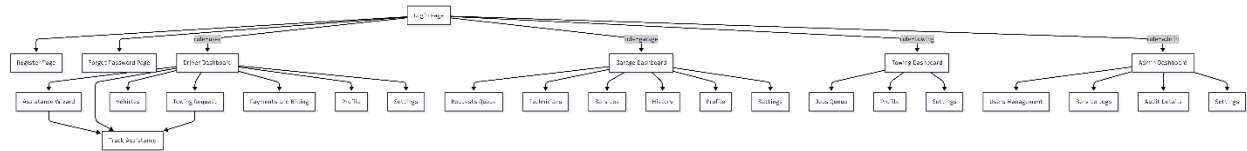
13.1 Password Reset



14. Screen Navigation Diagram

This diagram shows how the user moves between pages in the frontend.

14.1 Screen Navigation



15. Wireframes

The wireframes below show what the main screens will look like. They are simple mockups so that our team agrees on the layout before writing the real UI.

15.1 Login Page

[VAMP SYSTEM UI]

Sign In to VAMP
Vehicle Assistance &Emergency Management

Email Address
[user@test.com]

Password
[*****]

[Sign In]

15.2 Signup Page

```
+-----+
|                                             |
|               +-----+                   |
|               | [VAMP SYSTEM UI]         |
|               +-----+                   |
|                                             |
|               Create VAMP Account          |
|               Join the vehicle assistance  |
|               network                      |
|                                             |
|               +-----+                   |
|               | Full Name                 |
|               | [ Enter your name  ]     |
|               +-----+                   |
|                                             |
|               +-----+                   |
|               | Email Address             |
|               | [ Enter email address ]  |
|               +-----+                   |
|                                             |
|               +-----+                   |
|               | Password                  |
|               | [ Create password  ]     |
|               +-----+                   |
|                                             |
|               +-----+                   |
|               | [ Sign Up ]              |
|               +-----+                   |
|                                             |
|               Link: Already have an       |
|               account?                    |
|                                             |
+-----+
```

15.3 User Dashboard

```

+-----+-----+
| [VAMP LOGO]  User / Dashboard                                | () [ ] [] () |
+-----+-----+
| Dashboard                                           |               |
| Request Help                                       |               |
| Request Tow                                         |               |
| Track Assist                                       |               |
| My Vehicles                                         |               |
| Pay & Billing                                       |               |
+-----+-----+
|               |               |
| +-----+-----+               +-----+-----+               |
| | Hello, Alex Mercer! |               |               |
| | Need help on the road? Launch an emergency request. |               |
| | [Request Help] [Request Tow] |               |
| +-----+-----+               +-----+-----+               |
|               |               |
| QUICK DIAGNOSTIC ASSISTS                             |               |
| +-----+-----+ +-----+-----+ +-----+-----+ +-----+-----+ |
| | [Battery] | | [Engine] | | [Tire] | | [Collision] | |
| | Battery | | Engine | | Flat Tires | | Accidents & | |
| | Issues | | Problems | | Collisions | |
| +-----+-----+ +-----+-----+ +-----+-----+ +-----+-----+ |
|               |               |
| +-----+-----+ +-----+-----+ |
| | My Vehicles [Manage] | | Recent Incident Logs | | | | | |
| | +-----+-----+ | | +-----+-----+ | |
| | | [V] Tesla Model S | | | Incident | Garage | Fee | |
| | | [V] Toyota RAV4 | | | +-----+-----+ | |
| | +-----+-----+ | | Battery | Apex | $22 | |
| | +-----+-----+ | | +-----+-----+ | |
| +-----+-----+ +-----+-----+ |
|               |               |
| [AM] Alex Mencer |               |
+-----+-----+

```

15.4 Garage Dashboard

[VAMP LOGO] Garage Dashboard			() [] [] ()		
[] Overview [] Jobs [] Technicians [] Inventory [] Earnings	+-----+ +-----+ +-----+				
	INCIDENTS		ACTIVE REPAIR		COMPLETED
	0		0		2
	+-----+ +-----+ +-----+				
	ESTIMATED EARNINGS			TECHNICIAN WORKLOADS	
	\$3.30			Active Job Allocation	
	+-----+ +-----+ +-----+				
	INCIDENT TRENDS			URGENT BOARD	
	Weekly Intake & Clearance Rate			Nearby Unclaimed	
	[~~~~~~ Line Chart ~~~~~~]			Incident Calls	
+-----+ +-----+ +-----+					
[G] Garage Admin					

15.5 Towing Dashboard

[VAMP LOGO] Towing Dashboard		() [] [] ()		
[] Fleet Map [] Active Jobs [] Drivers [] Dispatch	+-----+ +-----+ +-----+			
	ACTIVE JOBS	COMPLETE TOWS	TOW EARNINGS	
	0 Jobs	--	\$2.04	
	+-----+ +-----+ +-----+			
	Collision Broadcast Board			
	[Open Live Tracker Grid]			
	+-----+ +-----+ +-----+			
	STATUS ALLOCATION		CLOSED RECOVERIES	
	Work Distribution		History Log	
	+-----+ +-----+ +-----+			
[T] Tow Dispatch				

15.6 Admin Dashboard

+-----+-----+ [VAMP LOGO] Admin / Dashboard () [] [] () +-----+-----+	
[] Overview [] User Control [] Global Logs	Platform Control Center
	Logged in as Super Admin [Control Accounts] [Feeds]
	+-----+ +-----+ +-----+ +-----+
	DRIVERS ACTIVE TOWING GROSS FEES
	ON NETWORK GARAGES OPERATIVES
	0 0 0 \$3.67
	+-----+ +-----+ +-----+ +-----+
	+-----+ +-----+
	ACCOUNT GROWTH INCIDENT CLASSIFICATIONS
	Registration Trajectory Emergency Log Categories
	[Drivers]
	[Tows] (0) Breakdowns
	[Workshops] (0) Battery
	+-----+ +-----+
	+-----+ +-----+
	LIVE TRAFFIC CONTROL Audit Operations Log
	Active Emergencies Feed Real-time system lifecycle
	+-----+ +-----+
+-----+-----+ [SA] Super Admin +-----+-----+	

16. UI Design

16.1 Screen Descriptions

Login Page — Email, password, and role selection. On success the user is taken to the dashboard for their role. On failure a toast message says “Invalid email or password.”

Register Page — Two steps. Step 1 collects name, email, password, phone, and role. Step 2 asks for role-specific fields (vehicle info for drivers, business name/hours/rate for garages and towing). A Terms and Conditions checkbox is needed to enable the “Create Account” button.

Driver Dashboard — Shows a welcome banner, an active request card if there is one, quick-action buttons for common issues (Battery, Engine, Tyre, Accident, Tow), a strip of the user’s vehicles, and a table of past requests.

Assistance Wizard — A multi-step form (Category → Symptoms → Details → Review). The details step includes vehicle selection, description, optional photo and voice-note buttons, and a map for the location.

Track Assistance — Shows a status timeline, the assigned provider and technician, an ETA, and a map with the provider’s location. When the job is completed it shows the estimated cost with an “Approve Cost” button.

Garage Requests Queue — A list of pending nearby requests with distance and cost. Each has an “Accept” button. Tabs at the top let the garage filter (All / Unclaimed / Active / Completed).

Admin Dashboard — Stats cards (users, garages, towing, active incidents, revenue), a revenue chart, a “Requests by Status” chart, and a recent-users table with suspend/reinstate controls.

16.2 UI Design System

- **Font:** Sans-serif system font (Inter, or the browser default sans-serif fallback).
- **Colours:** Primary blue for main actions, green for success/completed, amber for in-progress, red for errors, pink for admin accents.
- **Theme:** Light and dark themes, saved as a user preference.
- **Buttons:** Rounded corners, clear hover state, full-width on mobile.
- **Forms:** Inline validation with red text below the field.
- **Toasts:** Non-blocking notifications for success and error messages.

17. API Documentation

Base URL: *http://localhost:5000/api* **Authentication:** JWT bearer token in the *Authorization* header, or the *vamp-token* HTTP-only cookie. **Content-Type:** *application/json*

17.1 Authentication Endpoints

Method	Endpoint	Auth	Purpose
POST	/auth/register	No	Create a new account

Method	Endpoint	Auth	Purpose
POST	/auth/login	No	Log in and receive a JWT
GET	/auth/me	Yes	Get the current user's profile
PUT	/auth/profile	Yes	Update the current user's profile
POST	/auth/forgot-password	No	Request a password-reset link
POST	/auth/reset-password	No	Set a new password using a reset token
POST	/auth/logout	Yes	Clear the authentication cookie

17.2 User Management Endpoints

Method	Endpoint	Role	Purpose
GET	/users?role={role}	Authenticated	List users, optionally by role
PUT	/users/{id}	Admin	Suspend or reinstate a user
DELETE	/users/{id}	Self or Admin	Soft-delete an account

17.3 Request Management Endpoints

Method	Endpoint	Role	Purpose
GET	/requests?category={cat}	Authenticated	List requests (scoped by role)
POST	/requests	Driver	Create a new request
PUT	/requests/{id}	Garage / Towing / Admin	Update a request status
GET	/requests/export	Admin	Export all requests as CSV

17.4 Transaction Endpoints

Method	Endpoint	Role	Purpose
GET	/transactions	Authenticated	List transactions (scoped by role)
POST	/transactions	Driver	Create a payment transaction

17.5 Notification Endpoints

Method	Endpoint	Role	Purpose
GET	/notifications	Authenticated	List own notifications
PUT	/notifications/read	Authenticated	Mark all as read
DELETE	/notifications/{id}	Owner only	Delete one notification

18. Database Schema

The schema below shows the main columns of each table.

Table	Column	Type	Constraint
users	id	VARCHAR(255)	PK

Table	Column	Type	Constraint
	email	VARCHAR(255)	UNIQUE, NOT NULL
	password	VARCHAR(255)	NOT NULL
	role	VARCHAR(50)	NOT NULL
	name	VARCHAR(255)	NOT NULL
	phone	VARCHAR(50)	
	rate_per_km	DOUBLE	
	gps_lat, gps_lng	DOUBLE	
	suspended	BOOLEAN	DEFAULT FALSE
	deleted	BOOLEAN	DEFAULT FALSE
vehicles	id	VARCHAR(255)	PK
	make, model, year, plate, insurance	VARCHAR(255)	
	user_id	VARCHAR(255)	FK → users.id, NOT NULL
services	id	VARCHAR(255)	PK
	name, price	VARCHAR(255)	
	description	VARCHAR(1000)	
	garage_id	VARCHAR(255)	FK → users.id, NOT NULL
technicians	id	VARCHAR(255)	PK
	name, phone	VARCHAR(255)	
	status	VARCHAR(50)	DEFAULT 'available'
	garage_id	VARCHAR(255)	FK → users.id, NOT NULL
requests	id	VARCHAR(255)	PK
	user_id	VARCHAR(255)	NOT NULL
	category, status, payment_status	VARCHAR(50-100)	status DEFAULT 'pending', payment_status DEFAULT 'unpaid'
	symptoms, description	VARCHAR(2000)	
	gps_lat, gps_lng	DOUBLE	
	garage_id, towing_id	VARCHAR(255)	
	dispatch_cost, platform_commission, provider_share, distance	DOUBLE	
	created_at	VARCHAR(50)	ISO Instant
	deleted	BOOLEAN	DEFAULT FALSE

Table	Column	Type	Constraint
transactions	id	VARCHAR(255)	PK
	request_id, user_id, garage_id	VARCHAR(255)	
	amount, payment_method, status	VARCHAR(255)	status DEFAULT 'Successful'
	date	VARCHAR(50)	
notifications	id	VARCHAR(255)	PK
	user_id	VARCHAR(255)	NOT NULL
	type, title	VARCHAR(255)	
	message	VARCHAR(2000)	
	is_read	BOOLEAN	DEFAULT FALSE

19. Validation Rules

Field	Rule
Email	Required, valid format, unique on registration
Password	Required, minimum 6 characters
Name	Required, non-empty
Vehicle Year	Required, 4-digit year (e.g., 2020)
Vehicle Make / Model / Plate	Required
Request Category	Required, one of six predefined values
Request Symptoms	At least one must be selected
GPS Coordinates	Required before submitting a request
Technician Name / Phone	Required
Service Name / Price	Required
Payment Method	Required before payment
Reset Token	Must be valid and within 15-minute window
Terms and Conditions	Must be checked before account creation

20. Business Rules

ID	Rule
BR-01	A user account is identified by a unique email address.
BR-02	Passwords are never stored or transmitted in plain text. BCrypt is used.
BR-03	Dispatch Cost = (distance in km × provider's ratePerKM) ÷ 300
BR-04	Platform Commission = Dispatch Cost × 10%

ID	Rule
BR-05	Provider Share = Dispatch Cost – Platform Commission
BR-06	Request status must move in the defined order. Wrong transitions are rejected.
BR-07	Payment is only possible after status = <i>completed</i> .
BR-08	Suspended users cannot log in, but their history is kept.
BR-09	Soft-deleted users and requests are hidden from active views but kept in the database.
BR-10	Only admins can suspend, reinstate, or export system-wide data.
BR-11	A driver can only view and act on their own requests, vehicles, and transactions.
BR-12	A garage / towing operator can only see requests relevant to their type of service.

21. Security Requirements

- **SR-1:** Passwords are hashed with BCrypt before saving. Plain text passwords are never stored or logged.
- **SR-2:** Every authenticated API call needs a valid JWT, sent in the *Authorization: Bearer* header or the *vamp-token* HTTP-only cookie.
- **SR-3:** JWTs include the user's ID, email, and role. They expire after 7 days. Password reset tokens expire after 15 minutes.
- **SR-4:** Role-Based Access Control (RBAC) is enforced at the endpoint level (for example, *PUT /api/users/{id}* is admin-only).
- **SR-5:** CORS is restricted to the configured frontend origin.
- **SR-6:** The backend is stateless — no server-side session is stored for authentication.
- **SR-7:** In production, all traffic should use HTTPS / TLS.
- **SR-8:** Secrets like the JWT signing key and database password are kept outside the code, in environment variables.
- **SR-9:** All incoming data is validated with Bean Validation annotations on the DTOs.

22. Error Handling

The backend uses a central *GlobalExceptionHandler* to convert errors into JSON responses.

Exception	HTTP Status	Response
Validation failed	400	<i>{message: "Validation failed", errors: {...}}</i>
Business logic error	500	<i>{message: "<description>"}</i>
Uncaught exception	500	<i>{message: "An unexpected error occurred"}</i>
Wrong credentials	401	<i>{success: false, message: "Invalid email or password"}</i>

Exception	HTTP Status	Response
Not allowed	403	<code>{success: false, message: "Access denied"}</code>
Not found	404	<code>{success: false, message: "Resource not found"}</code>

On the frontend, these errors are shown to the user through a *ToastContext* as non-blocking notifications. Field-level errors are shown inline under the input.

23. Logging

- **LR-1** — Log all login attempts (success and failure) with time and email (never the password).
- **LR-2** — Log all status changes on a request (who changed it, from what to what, and when).
- **LR-3** — Log unhandled exceptions with a stack trace to the server console.
- **LR-4** — If SMTP is not configured, log the password-reset link to the console so the developer can still test.
- **LR-5** — Never log full JWTs or passwords.

In this version, logs are printed to the console. Structured file logging is planned as a future improvement.

24. Backup and Recovery

- **BR-1** — The MySQL database is backed up regularly using *mysqldump*.
- **BR-2** — A backup is taken before any change that might alter the schema (we use *ddl-auto=update*).
- **BR-3** — If data is lost, the system can be restored by loading the most recent SQL dump.
- **BR-4** — Soft-delete flags on users and requests give an application-level way to recover accidentally deleted records without restoring the whole database.
- **BR-5** — For this academic project, manual backups before demonstrations are enough. Automated backups are a future improvement.

25. Testing

We plan to test the system at three levels.

25.1 Unit Testing

- Test password hashing and JWT generation.

- Test dispatch cost, commission, and provider share calculations for different distances (including 0 km).
- Test DTO-to-entity and entity-to-DTO conversions.
- Test that validation rules reject invalid input.

25.2 Integration Testing

- Test the full auth flow: register → login → */auth/me*.
- Test the full request flow: create → accept → progress → complete.
- Test that a transaction correctly updates the *paymentStatus*.
- Test that a notification is created when a request is accepted.

25.3 System Testing

- End-to-end test of the whole assistance-request lifecycle across the frontend and backend.
- End-to-end test of the towing-request lifecycle.
- Cross-role test: a driver cannot access garage or admin pages, and vice versa.
- Responsive test: check the layout on desktop and mobile widths.

25.4 User Acceptance Testing

Scenario	Expected
A driver registers, adds a vehicle, and creates a battery-issue request	Request shows as <i>pending</i> for nearby garages
A garage accepts and assigns a technician	Driver sees updated status and technician contact on the Track page
Driver approves and pays	Transaction recorded, request marked <i>paid</i>
Admin suspends a garage account	Suspended garage cannot log in, but past data is still visible to admin
A user resets their password	Reset link works and the password is changed

26. Future Enhancements

- Full frontend + backend integration (some prototype pages currently use `localStorage`).
- Real-time updates using WebSocket instead of polling.
- Real file and audio uploads (currently just simulated flags in the database).
- Real payment gateway (Stripe or PayHere) instead of simulated payment.
- SMS and push notifications on top of in-app notifications.
- Save ratings and reviews in the backend (currently only in the frontend prototype).
- Automatic matching of the nearest available provider.
- Scheduled automatic database backups.
- Enforced HTTPS in production with a proper certificate.
- Multi-language support (English, Sinhala, Tamil).

27. Limitations

- Some frontend pages still use browser *localStorage* while we finish wiring them to the REST API.
- No WebSocket for real-time updates. The frontend refreshes when the user opens the page.
- Photo and voice-note attachments are only boolean flags in the database. Real files are not stored yet.
- Payment is simulated. No real money is moved.
- Distances are straight-line (Haversine) — not actual road distance.
- No background jobs. For example, an old *pending* request is not automatically cancelled.
- Logging is console-only. No file logging or dashboard yet.
- The project is developed within one semester by a small team, so there are no enterprise features like auto-scaling or multi-region deployment.

28. Appendix

28.1 Sample Test Accounts

For the demo, the database is pre-seeded with these accounts.

Role	Email	Password
Driver	user@test.com	password
Garage	garage@test.com	password
Towing	towing@test.com	password
Admin	admin@test.com	password

These accounts are only for local testing. They must not be used in production.

28.2 Tools Used

Purpose	Tool
Backend IDE	IntelliJ IDEA / VS Code
Frontend IDE	VS Code
API Testing	Postman / Thunder Client
Version Control	Git
Diagramming	Mermaid + custom PNGs
Database Management	MySQL Workbench
Design / Wireframes	Figma

28.3 How to Regenerate the Diagrams

All diagrams have the mermaid source shown in the document. You can copy any code block into the Mermaid Live Editor (<https://mermaid.live>) to view or export it. This lets you edit the diagrams without needing the original tools.

Group Contribution

The following table documents the specific contributions made by each group member to this Software Requirements Specification document. All members participated in review and feedback cycles.

Name	Student ID	Contribution
D.G.S. Wassana	39323	Frontend, Backend- Create and check "digital ID cards" (JWT Tokens) for users
A.K.M. Uyangoda	39282	Frontend, Backend- Make sure the system is safe and connects well with the frontend
H.M.H.S. Medawatta	39316	Frontend, Backend- Creating Towing Requests
H.W.K.S. Wijerathna	39140	Frontend, Backend- Errors, Notifications & Fake Data
D.D.S. De Zoysa	39314	Frontend, Backend- Updating Towing Requests
A.I. Amandi	38946	Backend- Login & Logout
H.I.W. Gamage	39036	Backend - User Registration
M.M.P. Deshitha	37924	Backend- Garages & Technicians Details
U.G.R. Ayesha	38910	Backend- User Profiles & Vehicles
D.M.R.I.B. Dissanayaka	39073	Backend- Payments

GitHub Frontend Repository: <https://github.com/kasunmihiran603/Resqdriver.git>

GitHub Backend Repository: <https://github.com/SahanWijerathna/Backend.git>